02 — Change-point detection

Scope. All findings describe the visible subset of pharmacy revenue (counter cash receipts, about 65%); see NB01 for the recording basis. Revenue is in LC, scaled local currency: every value is multiplied by one constant so that the sample mean is 120,000 (NB01). 25 December 2019 (an institutional credit settlement, per NB01) is excluded from the analysis below.

Signal preparation. Break detection and the bootstrap test operate on log revenue with day-of-week means removed, so that additive shifts read as percentage shifts, variance is steadier across regimes, and the Friday depression does not dominate residuals. Regime means and shifts are reported on the original revenue scale for interpretability.

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import ruptures as rpt
from joblib import Parallel, delayed
from pathlib import Path

sns.set_theme(style="whitegrid", context="notebook")
plt.rcParams["figure.figsize"] = (11, 4)
plt.rcParams["axes.titleweight"] = "bold"
```

In [2]:
```
DATA = Path("../data/daily_sales_final.csv")
df = pd.read_csv(DATA, parse_dates=["date"])
df = df.sort_values("date").set_index("date").asfreq("D")

# Per NB01, 25 December 2019 is an institutional credit settlement (a re
cording
# artefact from about a year of accumulated purchases paid on that date,
not a
# demand event). Excluded from level analyses and models throughout NB02
–NB05.
y_raw = df["revenue_scaled"].copy()
y_raw.loc[pd.Timestamp("2019-12-25")] = np.nan
y = y_raw.dropna()

# Working representation for break detection and the bootstrap test:
# log revenue converts additive shifts to percentage shifts and stabilis
es variance;
# day-of-week means are removed so the Friday depression does not domina
te residuals
# (which would make block-bootstrap resamples appear more variable than
they truly are).
log_y = np.log(y)
dow_means = log_y.groupby(log_y.index.dayofweek).transform("mean")
log_y_dow = log_y - dow_means

print(
```

```
    f"Working series: {len(y)} days from {y.index.min().date()} to {y.in
dex.max().date()}"
)
```

```
Working series: 469 days from 2019-12-15 to 2021-05-10
```

In [3]:
```
# Chow-style test at the a priori restrictions date (24 March 2020 — mat
ches
# the marker used in NB01). OLS on log revenue with day-of-week effects
removed,
# HAC (Newey-West) standard errors with a 21-lag bandwidth (three weekly
# cycles; a reasonable constant, not tuned to the results) to account fo
r
# positive serial correlation in daily revenue.
tau = pd.Timestamp("2020-03-24")
post_dummy = (log_y_dow.index >= tau).astype(int)
X = sm.add_constant(pd.DataFrame({"post": post_dummy}, index=log_y_dow.i
ndex))
res_chow = sm.OLS(log_y_dow, X).fit(cov_type="HAC", cov_kwds={"maxlags":
21})
shift_log = res_chow.params["post"]
shift_pct = (np.exp(shift_log) - 1) * 100

print(res_chow.summary().tables[1])
print(f"\nShift (log-scale): {shift_log:+.4f}")
print(f"Shift (percentage of pre-COVID mean): {shift_pct:+.1f}%")
print(f"t-stat (HAC): {res_chow.tvalues['post']:.2f}")
print(f"p-value (HAC): {res_chow.pvalues['post']:.2e}")
```

```
==============================================================================
====
                 coef    std err          z      P>|z|      [0.025      0.
975]
------------------------------------------------------------------------------
----
const          0.1938      0.021      9.026      0.000       0.152
0.236
post          -0.2411      0.037     -6.439      0.000      -0.314      -
0.168
==============================================================================
====

Shift (log-scale): -0.2411
Shift (percentage of pre-COVID mean): -21.4%
t-stat (HAC): -6.44
p-value (HAC): 1.20e-10
```

At the a priori restrictions date (24 March 2020), the estimated level shift on log revenue with day-of-week effects removed is −0.241 (equivalent to about a −21% change on the original revenue scale), with HAC standard errors and a 21-lag bandwidth. The HAC t-statistic is −6.44. This is a Chow-style test at an a priori date; the following section removes the dependence on the choice of date with a sup-Wald scan.

In [4]:
```
# Sup-Wald scan across every candidate break date in a 10%-trimmed windo
w.
# 10% trim (window starts 2020-02-01) is used rather than 15% (which wou
ld
# start 2020-03-03, the first observed day after the 25 Feb – 2 Mar miss
```

```
ing
# week) so the scan can consider candidate dates before the missing wee
k.
# Andrews (1993) critical values are slightly higher at 10% trim than at
15%. The peak of the surface is a test statistic;
# for a date estimate on this data, see the LS single-break block in the
K=3
# CI section below.
# HAC standard errors on each candidate break's regression.
trim = 0.10
n = len(log_y_dow)
lo = int(np.ceil(trim * n))
hi = int(np.floor((1 - trim) * n))
candidate_dates = log_y_dow.index[lo:hi]

results = []
for cand in candidate_dates:
    post = (log_y_dow.index >= cand).astype(int)
    X_c = sm.add_constant(pd.DataFrame({"post": post}, index=log_y_dow.i
ndex))
    r_c = sm.OLS(log_y_dow, X_c).fit(cov_type="HAC", cov_kwds={"maxlag
s": 21})
    results.append(
        {
            "tau": cand,
            "t_stat": r_c.tvalues["post"],
            "shift_log": r_c.params["post"],
        }
    )
sup = pd.DataFrame(results)
best = sup.loc[sup["t_stat"].abs().idxmax()]

print(
    f"Trimmed scan window: {candidate_dates[0].date()} to {candidate_dat
es[-1].date()}"
)
print(f"Peak location   : {best['tau'].date()}")
print(f"  t-stat (HAC) : {best['t_stat']:.2f}")
print(f"  shift (log)  : {best['shift_log']:+.4f}")
print(f"  shift (%)    : {(np.exp(best['shift_log']) - 1) * 100:+.1f}%")
print()
print("Andrews (1993) critical values for sup-Wald with 10% trim, single
break:")
print("  10% : 7.63   5% : 9.31   1% : 12.93")
print(f"Observed peak |t|^2 (sup-Wald equivalent): {best['t_stat'] **
2:.1f}")
```

```
Trimmed scan window: 2020-02-01 to 2021-03-23
Peak location   : 2020-02-22
  t-stat (HAC) : -7.80
  shift (log)  : -0.2581
  shift (%)    : -22.7%

Andrews (1993) critical values for sup-Wald with 10% trim, single break:
  10% : 7.63   5% : 9.31   1% : 12.93
Observed peak |t|^2 (sup-Wald equivalent): 60.8
```

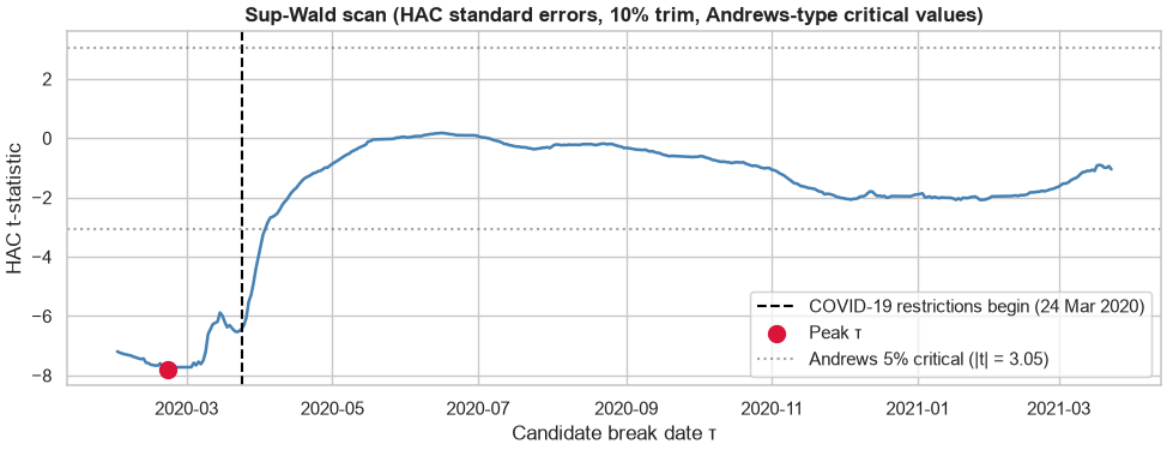
In [5]:

```
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(sup["tau"], sup["t_stat"], color="steelblue", linewidth=1.8)
ax.axvline(
```

```
    pd.Timestamp("2020-03-24"),
    color="black",
    linestyle="--",
    label="COVID-19 restrictions begin (24 Mar 2020)",
)
ax.scatter(
    best["tau"], best["t_stat"], color="crimson", s=100, zorder=5, label
="Peak τ"
)

# Andrews (1993) critical values at 10% trim, converted to |t| threshol
d.
crit_5pct = np.sqrt(9.31)
ax.axhline(-crit_5pct, color="grey", linestyle=":", alpha=0.7)
ax.axhline(
    crit_5pct,
    color="grey",
    linestyle=":",
    alpha=0.7,
    label=f"Andrews 5% critical (|t| = {crit_5pct:.2f})",
)

ax.set(
    xlabel="Candidate break date τ",
    ylabel="HAC t-statistic",
    title="Sup-Wald scan (HAC standard errors, 10% trim, Andrews-type cr
itical values)",
)
ax.legend(loc="best")
plt.tight_layout()
plt.savefig("../charts/nb02_sup_wald_scan.png", dpi=120, bbox_inches="ti
ght")
plt.show()
```



The sup-Wald scan across every candidate date in the 10%-trimmed window (1 February 2020 to 23 March 2021) rejects the null of no break at any conventional level: the peak sup-Wald statistic is 60.8, against the Andrews (1993) 1% critical value of 12.93 at 10% trim.

The location of the peak, however, is not a date estimate on this data. The HAC-studentised surface is flat either side of the 25 February – 2 March missing-data week (see the scan chart), so the peak (|t| = 7.80 at 22 February) sits on a plateau rather than at a sharp maximum. The peak of the HAC-studentised surface is a test

statistic, not the least-squares date estimate; a least-squares single-break date with a bootstrap confidence interval is reported later, in the K=3 CI section, for comparison with the K=3 break-date intervals reported there.

PELT sanity check on synthetic data

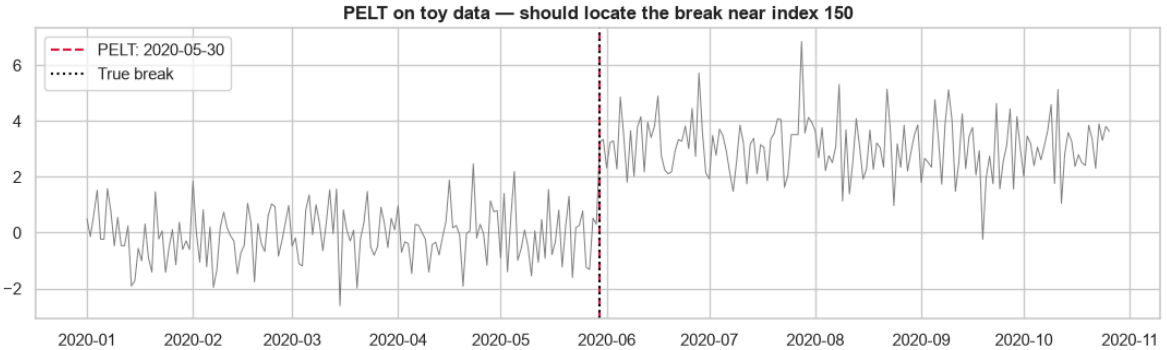
Before applying PELT to the revenue series, we verify it can locate a known break on a synthetic signal. The 300-day series below has a mean shift at index 150; PELT with an RBF kernel and moderate penalty should recover it.

In [6]:
```
np.random.seed(42)
n = 300
y_toy = np.concatenate(
    [
        np.random.normal(loc=0, scale=1, size=150),
        np.random.normal(loc=3, scale=1, size=150),
    ]
)
dates_toy = pd.date_range("2020-01-01", periods=n, freq="D")
```

In [7]:
```
algo = rpt.Pelt(model="rbf", jump=1).fit(y_toy)
detected = algo.predict(pen=10)
print(f"True break at index 150; PELT detected: {detected}")

fig, ax = plt.subplots(figsize=(11, 3.5))
ax.plot(dates_toy, y_toy, color="grey", linewidth=0.7)
for bp in detected[:-1]:
    ax.axvline(
        dates_toy[bp],
        color="crimson",
        linestyle="--",
        label=f"PELT: {dates_toy[bp].date()}",
    )
ax.axvline(dates_toy[150], color="black", linestyle=":", label="True break")
ax.set_title("PELT on toy data — should locate the break near index 150")
handles, labels = ax.get_legend_handles_labels()
by_label = dict(zip(labels, handles))
ax.legend(by_label.values(), by_label.keys())
plt.tight_layout()
plt.show()
```

```
True break at index 150; PELT detected: [150, 300]
```

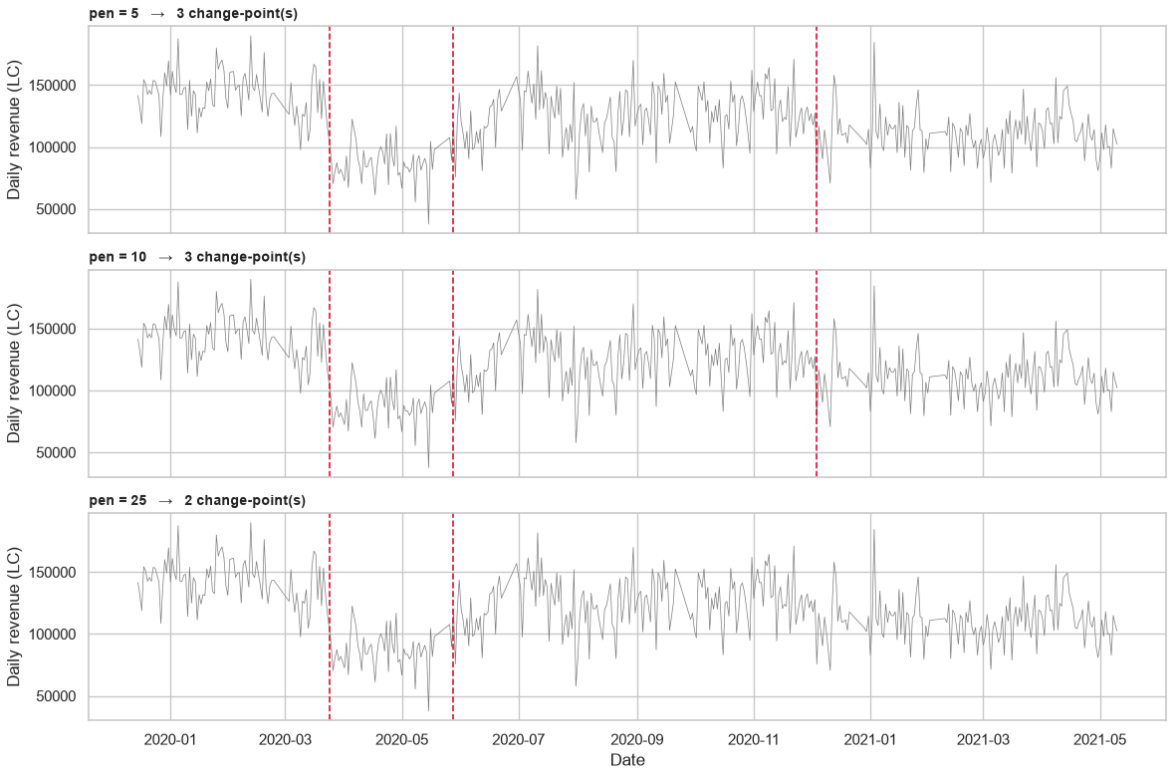


In [8]:
```
# Penalty sweep on the prepared signal (log revenue, DOW-deseasonalised,
# 25 Dec excluded). jump=1 gives day-accurate break dates; min_size=30 m
atches
# the Dynp default used in the bootstrap below.
signal = log_y_dow.values
for pen in [5, 10, 25, 50, 100]:
    algo = rpt.Pelt(model="rbf", jump=1, min_size=30).fit(signal)
    detected = algo.predict(pen=pen)[:-1]
    dates = [log_y_dow.index[i].date() for i in detected]
    print(f"pen={pen:>4}: {len(detected)} change points -> {dates}")
```

```
pen=   5: 3 change points -> [datetime.date(2020, 3, 24), datetime.date(20
20, 5, 28), datetime.date(2020, 12, 4)]
pen=  10: 3 change points -> [datetime.date(2020, 3, 24), datetime.date(20
20, 5, 28), datetime.date(2020, 12, 4)]
pen=  25: 2 change points -> [datetime.date(2020, 3, 24), datetime.date(20
20, 5, 28)]
pen=  50: 0 change points -> []
pen= 100: 0 change points -> []
```

Break dates on the prepared signal are identical at pen=5 and pen=10 (24 March, 28 May and 4 December 2020); pen=25 keeps the first two; pen=50 and above return none. The three dates at pen=10 form the primary structure used below. The penalty grid is a set of reasonable values, not tuned to the results; the bootstrap test later in the notebook asks whether the second and third breaks would arise from noise alone.

In [9]:
```
choices = [5, 10, 25]
fig, axes = plt.subplots(len(choices), 1, figsize=(12, 8), sharex=True)
for ax, pen in zip(axes, choices):
    algo = rpt.Pelt(model="rbf", jump=1, min_size=30).fit(signal)
    detected = algo.predict(pen=pen)[:-1]
    ax.plot(y.index, np.exp(log_y.values), color="grey", linewidth=0.5)
    for bp in detected:
        ax.axvline(log_y_dow.index[bp], color="crimson", linestyle="--",
linewidth=1.2)
    ax.set_title(
        f"pen = {pen}   →   {len(detected)} change-point(s)", fontsize=1
0, loc="left"
    )
    ax.set_ylabel("Daily revenue (LC)")
axes[-1].set_xlabel("Date")
plt.tight_layout()
plt.savefig("../charts/nb02_pelt_penalty_sweep.png", dpi=120, bbox_inche
s="tight")
plt.show()
```



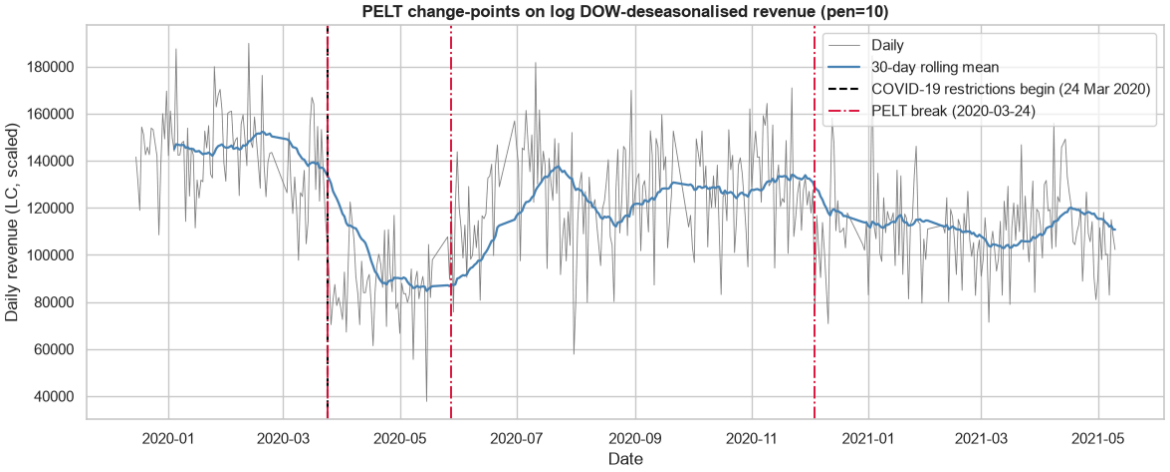
In [10]:

```
# PELT at pen=10, the middle of the grid above. min_size=30 observed day
s
# (about a month): no regime shorter than that can be found.
CHOSEN_PEN = 10
algo = rpt.Pelt(model="rbf", jump=1, min_size=30).fit(signal)
detected = algo.predict(pen=CHOSEN_PEN)[:-1]
detected_dates = [log_y_dow.index[bp] for bp in detected]

fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(y.index, y.values, color="grey", linewidth=0.6, label="Daily")
ax.plot(
    y.index,
    y.rolling(30, min_periods=20).mean(),
    color="steelblue",
    linewidth=1.6,
    label="30-day rolling mean",
)
ax.axvline(
    pd.Timestamp("2020-03-24"),
    color="black",
    linestyle="--",
    label="COVID-19 restrictions begin (24 Mar 2020)",
)
for i, d in enumerate(detected_dates):
    ax.axvline(
        d,
        color="crimson",
        linestyle="-.",
        linewidth=1.4,
        label=f"PELT break ({d.date()})" if i == 0 else None,
    )
ax.set(
    xlabel="Date",
    ylabel="Daily revenue (LC, scaled)",
    title=f"PELT change-points on log DOW-deseasonalised revenue (pen={C
HOSEN_PEN})",
```

```
)
ax.legend(loc="upper right")
plt.tight_layout()
plt.savefig("../charts/nb02_pelt_final.png", dpi=120, bbox_inches="tigh
t")
plt.show()

print(f"PELT (pen={CHOSEN_PEN}) break dates: {[d.date() for d in detecte
d_dates]}")
```




```
PELT (pen=10) break dates: [datetime.date(2020, 3, 24), datetime.date(202
0, 5, 28), datetime.date(2020, 12, 4)]
```

Next. Test whether the shift persists into 2021, characterise the four regimes, then validate the number of breaks with a sequential block bootstrap. The bootstrap set-up first checks that Dynp with L2 cost recovers the PELT dates.

In [11]:
```
# Persistence test: baseline (Dec 2019–Feb 2020, excl. 25 Dec) vs termin
al (Feb–May 2021).
# HAC-adjusted OLS on log revenue. Two variants reported: with and witho
ut
# excluding Ramadan 2021 from the terminal window, since Ramadan 2021 si
ts
# inside the Feb–May 2021 range.
baseline_start, baseline_end = "2019-12-15", "2020-02-29"
terminal_start, terminal_end = "2021-02-10", "2021-05-10"
ramadan_2021 = pd.date_range("2021-04-13", "2021-05-12")


def hac_shift(baseline_series, terminal_series, label):
    combined = pd.concat([baseline_series, terminal_series])
    post_ind = pd.Series(
        [0] * len(baseline_series) + [1] * len(terminal_series), index=c
ombined.index
    )
    X_p = sm.add_constant(pd.DataFrame({"post": post_ind}, index=combine
d.index))
    res = sm.OLS(combined, X_p).fit(cov_type="HAC", cov_kwds={"maxlags":
21})
    shift_pct = (np.exp(res.params["post"]) - 1) * 100
    print(f"{label}:")
    print(
        f"  Baseline mean : {np.exp(baseline_series).mean():,.0f} LC   "
        f"Terminal mean : {np.exp(terminal_series).mean():,.0f} LC"
    )
```

```
    print(
        f"  Log shift {res.params['post']:+.4f}   Percentage shift {shif
t_pct:+.1f}%   "
        f"HAC t {res.tvalues['post']:.2f}   HAC p {res.pvalues['post']:.
2e}"
    )
    return res


log_baseline = np.log(y.loc[baseline_start:baseline_end])
log_terminal_all = np.log(y.loc[terminal_start:terminal_end])
_ = hac_shift(log_baseline, log_terminal_all, "Full terminal window")

print()
log_terminal_no_ram = log_terminal_all.drop(
    log_terminal_all.index.intersection(ramadan_2021), errors="ignore"
)
_ = hac_shift(log_baseline, log_terminal_no_ram, "Terminal window excl.
Ramadan 2021")
```

```
Full terminal window:
  Baseline mean : 146,308 LC   Terminal mean : 109,587 LC
  Log shift -0.2940   Percentage shift -25.5%   HAC t -9.40   HAC p 5.41e-
21

Terminal window excl. Ramadan 2021:
  Baseline mean : 146,308 LC   Terminal mean : 110,051 LC
  Log shift -0.2903   Percentage shift -25.2%   HAC t -8.44   HAC p 3.24e-
17
```

Revenue averaged 146,308 LC/day in the pre-COVID baseline (December 2019 to February 2020, excluding the 25 December 2019 credit-settlement day) and 109,587 LC/day over the terminal window, 10 February to 10 May 2021 (a reasonable choice of window, made before the results and not tuned to them). The log-scale shift is about −0.29, equivalent to about −25.5% (on the log scale; the ratio of the arithmetic means is −25.1%), with HAC t ≈ −9.4. Excluding Ramadan 2021 from the terminal window gives about −25.2%. NB01 reports about −24% for the same shift as a ratio of arithmetic means over January–May 2021; both the window and the scale differ. The shift persists more than a year after the initial break; the sample ends before any full recovery could be observed, so this notebook can only characterise the persistence within the observed window.

In [12]:
```
# PELT on the post-restrictions subset of the prepared signal.
post_covid_mask = log_y_dow.index >= pd.Timestamp("2020-03-24")
post_covid_signal = log_y_dow[post_covid_mask].values
post_covid_index = log_y_dow.index[post_covid_mask]

for pen in [5, 10, 25, 50]:
    algo = rpt.Pelt(model="rbf", jump=1, min_size=30).fit(post_covid_sig
nal)
    detected = algo.predict(pen=pen)[:-1]
    dates = [post_covid_index[i].date() for i in detected]
    print(f"pen={pen:>3}: {len(detected)} breaks → {dates}")
```

```
pen=  5: 2 breaks → [datetime.date(2020, 5, 28), datetime.date(2020, 12,
4)]
```

```
pen= 10: 2 breaks → [datetime.date(2020, 5, 28), datetime.date(2020, 12,
4)]
pen= 25: 1 breaks → [datetime.date(2020, 5, 28)]
pen= 50: 0 breaks → []
```

PELT run on the post-restrictions subset picks two breaks at pen=10 (28 May and 4 December 2020), matching the second and third breaks from the full-series analysis. This is descriptive evidence that the post-shock period is not a flat plateau but instead has additional structure. The bootstrap section below evaluates whether that additional structure is supported by the data or would arise from noise alone.

In [13]:
```
# Regime boundaries taken from PELT at pen=10 on the prepared signal
# (log revenue, DOW-deseasonalised, 25 Dec excluded). Three breaks:
# 2020-03-24, 2020-05-28, 2020-12-04.

break_dates = [
    pd.Timestamp("2020-03-24"),
    pd.Timestamp("2020-05-28"),
    pd.Timestamp("2020-12-04"),
]
one_day = pd.DateOffset(days=1)
boundaries = [
    ("Pre-COVID", y.index.min(), break_dates[0] - one_day),
    ("Acute crash", break_dates[0], break_dates[1] - one_day),
    ("Partial rebound", break_dates[1], break_dates[2] - one_day),
    ("Lower plateau", break_dates[2], y.index.max()),
]

regimes = []
prev_mean = None
for name, start, end in boundaries:
    window = y.loc[start:end]
    m = window.mean()
    pct_vs_prev = None if prev_mean is None else (m / prev_mean - 1) * 1
00
    regimes.append(
        {
            "regime": name,
            "start": start.date(),
            "end": end.date(),
            "n_days": len(window),
            "mean_LC": f"{m:,.0f}",
            "% vs previous": "—" if pct_vs_prev is None else f"{pct_vs_p
rev:+.1f}",
        }
    )
    prev_mean = m

reg_df = pd.DataFrame(regimes)
print(reg_df.to_string(index=False))
```

```
        regime      start        end  n_days mean_LC % vs previous
     Pre-COVID 2019-12-15 2020-03-23      92 143,106             —
   Acute crash 2020-03-24 2020-05-27      58  87,209         -39.1
Partial rebound 2020-05-28 2020-12-03     176 125,589         +44.0
  Lower plateau 2020-12-04 2021-05-10     143 110,406         -12.1
```

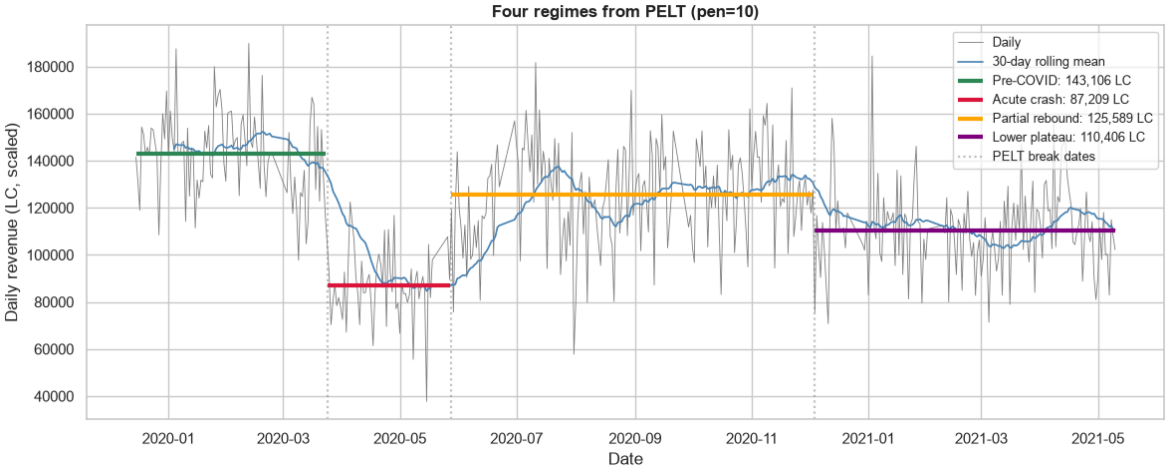
In [14]:
```
fig, ax = plt.subplots(figsize=(12, 5))
```

```
ax.plot(y.index, y.values, color="grey", linewidth=0.6, label="Daily")
ax.plot(
    y.index,
    y.rolling(30, min_periods=20).mean(),
    color="steelblue",
    linewidth=1.2,
    label="30-day rolling mean",
)

colours = ["seagreen", "crimson", "orange", "purple"]
for (name, start, end), c in zip(boundaries, colours):
    m = y.loc[start:end].mean()
    ax.hlines(
        m,
        xmin=pd.Timestamp(start),
        xmax=pd.Timestamp(end),
        colors=c,
        linewidth=3,
        label=f"{name}: {m:,.0f} LC",
    )

for i, (name, start, _) in enumerate(boundaries[1:]):
    ax.axvline(
        pd.Timestamp(start),
        color="grey",
        linestyle=":",
        alpha=0.5,
        label="PELT break dates" if i == 0 else None,
    )

ax.set(
    xlabel="Date",
    ylabel="Daily revenue (LC, scaled)",
    title="Four regimes from PELT (pen=10)",
)
ax.legend(loc="upper right", fontsize=9)
plt.tight_layout()
plt.savefig("../charts/nb02_four_regimes.png", dpi=120, bbox_inches="tig
ht")
plt.show()
```



The four regimes are:

- **Pre-COVID** (15 Dec 2019 – 23 Mar 2020, 92 days): mean 143,106 LC/day.

- **Acute crash** (24 Mar – 27 May 2020, 58 days): mean 87,209 LC/day, about 39% below pre-COVID.
- **Partial rebound** (28 May – 3 Dec 2020, 176 days): mean 125,589 LC/day, about 44% above the crash but still about 12% below pre-COVID.
- **Lower plateau** (4 Dec 2020 – 10 May 2021, 143 days): mean 110,406 LC/day, about 12% below the rebound and about 23% below pre-COVID.

The percentages above are descriptive summaries of the regimes selected by PELT from the same data. The bootstrap test in the next section evaluates whether the multi-regime structure — beyond a single break at the pandemic onset — is supported by the data.

Bootstrap test of the number of breaks

PELT identifies three breaks. Because those breaks were chosen from the same data, standard tests on the resulting regimes would be invalid. The bootstrap below simulates data under the null that additional breaks are not real, reruns the same break search on every simulated series, and compares the observed reduction in the residual sum of squares with the simulated distribution. Because the search runs inside every bootstrap replicate, the p-value accounts for the selection.

The test operates on log revenue with day-of-week means removed and 25 December 2019 excluded. Sequential tests move from K=0 vs K=1 through K=3 vs K=4 to show where the evidence stops. Sensitivity to the short-memory assumption is checked with block lengths of 7, 14 and 28 days. p-values use the standard formula p = (1 + #{boot ≥ observed}) / (B + 1) with B = 999.

Note on interpretation: residuals from the K-break fit still contain any real extra structure, which makes the test conservative if anything: the safe direction of bias.

In [15]:
```
# Bootstrap settings.
from pharmacy_covid_analysis.bootstrap import (
    MIN_SEG_LEN,
    fit_ssr,
    null_fit,
    run_sequential_bootstrap,
    one_break_date_iter,
)

BOOTSTRAP_B = 999
BOOTSTRAP_ALPHA = 0.05
BOOTSTRAP_PRIMARY_BLOCK = 14
BOOTSTRAP_BLOCK_LENGTHS = [7, 14, 28]
BOOTSTRAP_MAX_K = 4
BOOTSTRAP_SEED = 42


# Cross-check: does Dynp (L2 cost, mean-only) recover the same break dat
es
# as PELT (RBF cost, non-parametric) on the same signal? Agreement means
the
```

```
# bootstrap below — which uses Dynp — is validating the PELT breaks.
dynp_algo = rpt.Dynp(model="l2", jump=1, min_size=MIN_SEG_LEN).fit(log_y
_dow.values)
dynp_bkps = dynp_algo.predict(n_bkps=3)[:-1]
pelt_algo = rpt.Pelt(model="rbf", jump=1, min_size=MIN_SEG_LEN).fit(log_
y_dow.values)
pelt_bkps = pelt_algo.predict(pen=10)[:-1]
print("Dynp K=3 break dates (l2):", [log_y_dow.index[i].date() for i in
dynp_bkps])
print("PELT pen=10 break dates (rbf):", [log_y_dow.index[i].date() for i
in pelt_bkps])
```

```
Dynp K=3 break dates (l2): [datetime.date(2020, 3, 24), datetime.date(202
0, 5, 28), datetime.date(2020, 12, 4)]
PELT pen=10 break dates (rbf): [datetime.date(2020, 3, 24), datetime.date
(2020, 5, 28), datetime.date(2020, 12, 4)]
```

In [16]:

```
# Run sequential bootstrap tests: K=0 vs K=1, K=1 vs K=2, K=2 vs K=3, K=
3 vs K=4.
# For each sequential comparison, run under three block lengths.
# WARNING: with B=999 and jump=1 Dynp, this cell takes ~15-25 min with p
arallel joblib.
signal = log_y_dow.values

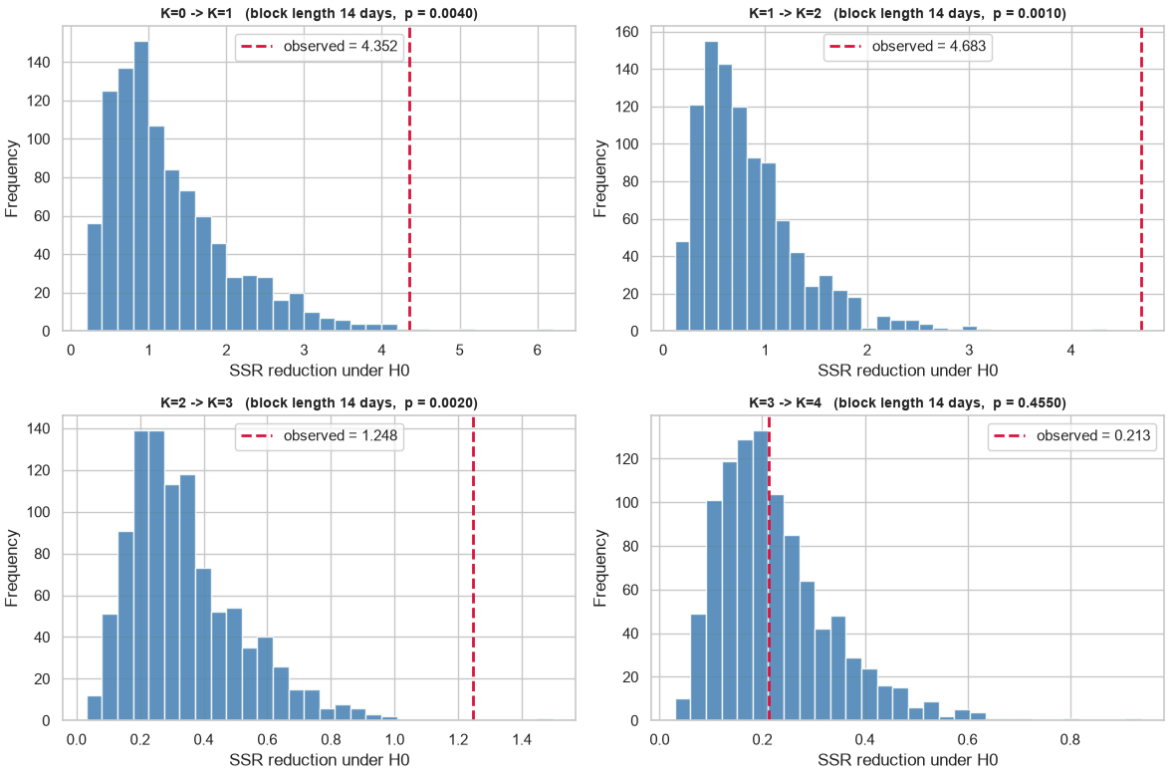
sequential_results = {}
for K0, K1 in [(0, 1), (1, 2), (2, 3), (3, 4)]:
    sequential_results[(K0, K1)] = {}
    for bl in BOOTSTRAP_BLOCK_LENGTHS:
        obs_stat, boot_stats, boot_bkps = run_sequential_bootstrap(
            signal, K0, K1, BOOTSTRAP_B, bl, BOOTSTRAP_SEED
        )
        p_value = (1 + np.sum(boot_stats >= obs_stat)) / (BOOTSTRAP_B +
1)
        sequential_results[(K0, K1)][bl] = {
            "obs_stat": obs_stat,
            "boot_stats": boot_stats,
            "boot_bkps": boot_bkps,
            "p_value": p_value,
        }

# Present as a table
rows = []
for (K0, K1), by_bl in sequential_results.items():
    for bl, r in by_bl.items():
        rows.append(
            {
                "H0 -> H1": f"K={K0} -> K={K1}",
                "block length (days)": bl,
                "observed SSR reduction": r["obs_stat"],
                "p-value": r["p_value"],
            }
        )
seq_df = pd.DataFrame(rows)
with pd.option_context("display.float_format", "{:.4f}".format):
    print(seq_df.to_string(index=False))
```

```
  H0 -> H1  block length (days)  observed SSR reduction  p-value
K=0 -> K=1                    7                  4.3521   0.0010
K=0 -> K=1                   14                  4.3521   0.0040
K=0 -> K=1                   28                  4.3521   0.0530
K=1 -> K=2                    7                  4.6834   0.0010
K=1 -> K=2                   14                  4.6834   0.0010
K=1 -> K=2                   28                  4.6834   0.0040
K=2 -> K=3                    7                  1.2482   0.0010
K=2 -> K=3                   14                  1.2482   0.0020
K=2 -> K=3                   28                  1.2482   0.0090
K=3 -> K=4                    7                  0.2127   0.3070
K=3 -> K=4                   14                  0.2127   0.4550
K=3 -> K=4                   28                  0.2127   0.5460
```

In [17]:

```python
# Histogram of the bootstrap null distributions with the observed statis
tic marked.
# Shows the four sequential tests at the primary block length (14 days).
primary_bl = 14
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
for ax, (K0, K1) in zip(axes.flatten(), [(0, 1), (1, 2), (2, 3), (3,
4)]):
    r = sequential_results[(K0, K1)][primary_bl]
    ax.hist(r["boot_stats"], bins=30, color="steelblue", edgecolor="whit
e", alpha=0.85)
    ax.axvline(
        r["obs_stat"],
        color="crimson",
        linestyle="--",
        linewidth=2,
        label=f"observed = {r['obs_stat']:.3f}",
    )
    ax.set_title(
        f"K={K0} -> K={K1}   (block length {primary_bl} days,  p = {r['p
_value']:.4f})",
        fontsize=10,
    )
    ax.set_xlabel("SSR reduction under H0")
    ax.set_ylabel("Frequency")
    ax.legend()
plt.tight_layout()
plt.savefig(
    "../charts/nb02_bootstrap_null_distributions.png", dpi=120, bbox_inc
hes="tight"
)
plt.show()
```



The primary block length is 14 days (two weekly cycles; a reasonable constant, not tuned to the results); 7 and 28 are sensitivity checks. At those settings:

- **K=0 → K=1** (is there any break at all?): p ≈ 0.001–0.004 at block lengths 7 and 14, p ≈ 0.053 at 28. This does not mean the first break is weak. Under the K=0 null the residuals contain the entire post-COVID drop, and 28-day blocks preserve those long deviations into the simulated series, inflating the null distribution and making this step conservative. The sup-Wald test above with valid Andrews critical values is the cleaner test for step one.
- **K=1 → K=2**: p ≈ 0.001–0.004 across block lengths. Rejects the null.
- **K=2 → K=3**: p ≈ 0.001–0.009 across block lengths. Rejects the null.
- **K=3 → K=4**: p ≈ 0.31–0.55 across block lengths. Does not reject; the four-regime structure is where the evidence stops.

Taken together, the data are consistent with three breaks, a four-regime structure, under post-selection-corrected tests. A fifth regime is not supported.

In [18]:
```
# Break-date confidence intervals for the K=3 structure.
# Semi-parametric bootstrap: fit K=3 on observed, block-bootstrap the re
siduals,
# rerun K=3 fit on each bootstrap sample, record the break dates.
_, obs_bkps3 = fit_ssr(signal, 3)
fitted3, residuals3, _ = null_fit(signal, 3)

ss = np.random.SeedSequence(BOOTSTRAP_SEED + 1).spawn(BOOTSTRAP_B)
boot_bkps_all = Parallel(n_jobs=-1)(
    delayed(one_break_date_iter)(cs, fitted3, residuals3, 3, primary_bl)
for cs in ss
)
boot_bkps_arr = np.array(boot_bkps_all)  # (B, 3)

# Sort each row so the k-th column corresponds to the k-th earliest brea
```

```
k
boot_bkps_arr = np.sort(boot_bkps_arr, axis=1)
obs_bkps3_sorted = sorted(obs_bkps3)

ci_rows = []
for j, obs_idx in enumerate(obs_bkps3_sorted):
    lower = int(np.quantile(boot_bkps_arr[:, j], 0.025))
    upper = int(np.quantile(boot_bkps_arr[:, j], 0.975))
    ci_rows.append(
        {
            "break_index": j + 1,
            "observed_date": log_y_dow.index[obs_idx].date(),
            "95% CI lower": log_y_dow.index[max(0, lower)].date(),
            "95% CI upper": log_y_dow.index[min(len(log_y_dow) - 1, uppe
r)].date(),
        }
    )
ci_df = pd.DataFrame(ci_rows)
print("K=3 break-date 95% bootstrap confidence intervals (block length 1
4 days):")
print(ci_df.to_string(index=False))

# Single-break (K=1) LS date + bootstrap CI for comparison. Reported her
e rather
# than beside the sup-Wald scan because it uses the same bootstrap machi
nery as
# the K=3 CIs above.
_, bkp1 = fit_ssr(signal, 1)
fitted1, residuals1, _ = null_fit(signal, 1)
ss1 = np.random.SeedSequence(BOOTSTRAP_SEED + 2).spawn(BOOTSTRAP_B)
b1 = Parallel(n_jobs=-1)(
    delayed(one_break_date_iter)(cs, fitted1, residuals1, 1, primary_bl)
for cs in ss1
)
b1_arr = np.array([b[0] for b in b1])
b1_lo, b1_hi = np.quantile(b1_arr, [0.025, 0.975]).astype(int)
print()
print("Single-break (K=1) LS date + 95% bootstrap CI (block length 14 da
ys):")
print(
    f"  LS date : {log_y_dow.index[bkp1[0]].date()}   "
    f"95% CI : {log_y_dow.index[b1_lo].date()} to {log_y_dow.index[b1_h
i].date()}"
)
```

```
K=3 break-date 95% bootstrap confidence intervals (block length 14 days):
 break_index observed_date 95% CI lower 95% CI upper
           1    2020-03-24   2020-03-23   2020-03-25
           2    2020-05-28   2020-05-26   2020-05-30
           3    2020-12-04   2020-10-08   2021-01-18
Single-break (K=1) LS date + 95% bootstrap CI (block length 14 days):
  LS date : 2020-03-22   95% CI : 2020-02-10 to 2020-06-15
```

The confidence intervals here quantify sampling variability given a chosen model: they say how precisely the search locates each break if it exists at that structural position. They do not quantify specification uncertainty, which is a separate issue in this data: the LS single-break model puts the break at 22 March 2020 with a 95% bootstrap CI stretching roughly from mid-February to mid-June, while the three-

break model puts the first break sharply on 24 March 2020 with a 95% CI of 23–25 March.

The very wide K=1 interval (about four months) is a diagnostic in its own right. It is consistent with the single-break model being misspecified, either because the process has more than one break or because the transition was gradual rather than a sharp step; it is a further reason to prefer the K=3 structure. The LS K=1 point estimate (22 March) is also much closer to the 24 March restrictions marker than the sup-Wald peak location (22 February). NB01's finding of about –10% for revenue over 3–23 March 2020 vs the January–February 2020 average suggests the pre-restrictions period was already softer than the pre-COVID baseline, so a mean-shift model that only represents steps has to place a single step somewhere in that softening window. The tight CI on the first K=3 break should be read as "given a three-break specification, the sharp step is precisely at 24 March", not as "the pandemic-related decline began sharply on 24 March".

The second K=3 break has a similarly tight CI (26–30 May 2020). One limitation: the 19–25 May missing-data week contains the end of Ramadan 2020 (23 May) and Eid al-Fitr (24–25 May), and the CI begins on the first observed day after that gap. The data cannot separate the end of Ramadan or Eid from the beginning of the partial rebound as drivers of the transition.

The third K=3 break has a much wider CI (October 2020 to January 2021), so the "lower plateau" regime is better read as beginning sometime in late 2020 or early 2021 than as a sharp transition on 4 December.

Robustness: Ramadan and Eid controls

Ramadan is a month-long religious observance (not itself a public holiday) that reshapes daily activity; Eid al-Fitr and Eid al-Adha are the public holidays that follow Ramadan and the pilgrimage. Ramadan 2020 (24 April – 23 May) sits inside the acute-crash regime, so a single "post-COVID" dummy would confound the two effects. The test below estimates the Ramadan and Eid effects jointly with the three-break regime structure — each dummy measured within its regime rather than against the whole post-COVID average — then subtracts only the Ramadan + Eid component and reruns PELT.

In [19]:
```
# Religious-calendar robustness. Ramadan (religious observance, not a pu
blic
# holiday) and Eid al-Adha (public holiday) effects are estimated jointl
y with the
# three-break regime structure (so each dummy is measured within its reg
ime),
# then only the Ramadan + Eid component is subtracted before rerunning t
he
# break search. A stability check then re-estimates the effects at the
# recovered break dates and reruns the search.
ramadan_2020 = pd.date_range("2020-04-24", "2020-05-23")
ramadan_2021_days = pd.date_range("2021-04-13", "2021-05-12")
```

```
ramadan_days = ramadan_2020.union(ramadan_2021_days)
# Eid al-Fitr 2020 (24-25 May) falls inside the 19-25 May missing-data w
eek
# per NB01, and Eid al-Fitr 2021 (13-14 May) falls after the 10 May samp
le end,
# so the Eid dummy is identified only by Eid al-Adha 2020 (31 July, 1 Au
gust).
eid_days = pd.to_datetime(
    ["2020-05-24", "2020-05-25", "2020-07-31", "2020-08-01", "2021-05-1
3", "2021-05-14"]
)

idx = log_y_dow.index
regime = sum((idx >= b).astype(int) for b in break_dates)  # 0, 1, 2, 3
X_joint = sm.add_constant(
    pd.DataFrame(
        {
            **{f"regime_{k}": (regime == k).astype(int) for k in (1, 2,
3)},
            "ramadan": idx.isin(ramadan_days).astype(int),
            "eid": idx.isin(eid_days).astype(int),
        },
        index=idx,
    )
)
res_joint = sm.OLS(log_y_dow, X_joint).fit(cov_type="HAC", cov_kwds={"ma
xlags": 21})

holiday_adj = (
    log_y_dow
    - res_joint.params["ramadan"] * X_joint["ramadan"]
    - res_joint.params["eid"] * X_joint["eid"]
)
algo_adj = rpt.Pelt(model="rbf", jump=1, min_size=30).fit(holiday_adj.va
lues)
dates_adj = [idx[i].date() for i in algo_adj.predict(pen=10)[:-1]]
_, dynp_bkps_adj = fit_ssr(holiday_adj.values, 3)
dates_adj_dynp = [idx[i].date() for i in dynp_bkps_adj]

n_eid_in_sample = X_joint["eid"].sum()
print(
    f"Ramadan coefficient (log): {res_joint.params['ramadan']:+.4f}   HA
C t: {res_joint.tvalues['ramadan']:.2f}"
)
print(
    f"  On the original scale: exp(beta) - 1 = {(np.exp(res_joint.params
['ramadan']) - 1) * 100:+.1f}%"
)
print(
    f"Eid coefficient (log): {res_joint.params['eid']:+.4f}   (identifie
d by {n_eid_in_sample} in-sample days)"
)
print()
print("PELT break dates with only the religious-calendar component remov
ed (pen=10):")
print(f"  {dates_adj}")
print(f"Dynp K=3 break dates on the same series: {dates_adj_dynp}")

# Stability check: re-estimate the religious-calendar effects at the new
break dates.
```

```
new_break_dates = [pd.Timestamp(str(d)) for d in dates_adj_dynp]
regime_new = sum((idx >= b).astype(int) for b in new_break_dates)
X_check = sm.add_constant(
    pd.DataFrame(
        {
            **{f"regime_{k}": (regime_new == k).astype(int) for k in (1,
2, 3)},
            "ramadan": idx.isin(ramadan_days).astype(int),
            "eid": idx.isin(eid_days).astype(int),
        },
        index=idx,
    )
)
res_check = sm.OLS(log_y_dow, X_check).fit(cov_type="HAC", cov_kwds={"ma
xlags": 21})
holiday_adj_check = (
    log_y_dow
    - res_check.params["ramadan"] * X_check["ramadan"]
    - res_check.params["eid"] * X_check["eid"]
)
_, dynp_bkps_check = fit_ssr(holiday_adj_check.values, 3)
dates_check = [idx[i].date() for i in dynp_bkps_check]
print(
    f"Stability check (holiday effects re-estimated at new break dates):
{dates_check}"
)
```

```
Ramadan coefficient (log): -0.0332   HAC t: -1.31
  On the original scale: exp(beta) - 1 = -3.3%
Eid coefficient (log): -0.5143   (identified by 2 in-sample days)

PELT break dates with only the religious-calendar component removed (pen=1
0):
  [datetime.date(2020, 3, 24), datetime.date(2020, 5, 28), datetime.date(2
020, 12, 4)]
Dynp K=3 break dates on the same series: [datetime.date(2020, 3, 24), date
time.date(2020, 5, 28), datetime.date(2020, 12, 4)]
Stability check (holiday effects re-estimated at new break dates): [dateti
me.date(2020, 3, 24), datetime.date(2020, 5, 28), datetime.date(2020, 12,
4)]
```

Three observations from the joint religious-calendar check:

- **Break dates are unchanged.** With Ramadan and Eid estimated alongside the three regime dummies and only the religious-calendar component subtracted before rerunning the break search, PELT still picks 24 March, 28 May and 4 December 2020 (Dynp K=3 gives the same three dates). The stability check that re-estimates the effects at the recovered break dates returns the same dates again.
- **The Ramadan effect is small and not statistically significant.** In the joint model, the Ramadan-period dummy has a coefficient of about −0.03 on the log scale (about −3% on the original scale, HAC t = −1.31).
- **The Eid coefficient is identified by two days.** Eid al-Fitr 2020 (24–25 May) falls inside the 19–25 May missing-data week per NB01, and Eid al-Fitr 2021 (13–14 May) falls after the 10 May sample end. Only Eid al-Adha 2020 (31 July and 1 August) is in-sample, so the coefficient of about −0.5 on the log scale

summarises what happened on those two dates. It should not be read as a general Eid effect.

Summary

All findings describe counter cash receipts (about 65% of revenue; see NB01), analysed as log revenue with day-of-week means removed and 25 December 2019 excluded.

Existence. A sup-Wald scan with HAC standard errors rejects "no break" (peak statistic 60.8 against a 1% critical value of 12.93 at 10% trimming). The Chow-style test at the a priori 24 March date gives a −21.4% shift (HAC t = −6.44). The scan's peak location is not a date estimate: the surface is flat around the 25 February – 2 March missing week.

Number. Four regimes. A block bootstrap that reruns the break search on every simulated series (B = 999; primary block length 14 days, with 7 and 28 as sensitivity checks) rejects K=0→1 (at block lengths 7 and 14; p = 0.053 at 28, see above), 1→2 and 2→3, and does not reject K=3→4.

Timing. PELT and the exact search agree on 24 March 2020, 28 May 2020 and 4 December 2020. Given the three-break model, the first two dates are precise (95% intervals of 23–25 March and 26–30 May); the third is not (October 2020 to January 2021). The second interval starts on the first day after the 19–25 May missing week, which contains the end of Ramadan and Eid al-Fitr, so the data cannot separate these from the easing of restrictions as causes of the rebound. A single-break model dates the shift to 22 March with a four-month interval, consistent with a decline that began before the restrictions (NB01: −10% over 3–23 March) followed by a sharp step on 24 March.

Size. Regime means: pre-COVID 143,106 LC/day; acute crash 87,209 (−39%); partial rebound 125,589 (−12% versus pre-COVID); lower plateau 110,406 (−23%). Revenue in February–May 2021 remained about 25% below the pre-COVID baseline (HAC t = −9.40), unchanged when Ramadan 2021 is excluded.

Robustness. Estimating Ramadan (religious observance) and Eid al-Adha (public holiday) effects jointly with the regime structure leaves the break dates unchanged. The Ramadan effect is small and not significant (about −3%); the Eid coefficient rests on two days (Eid al-Adha 2020) and is not a general estimate.

The intervals reflect sampling variability within a given model, not uncertainty about which model is right.

On the weekly-pattern assumption. The day-of-week correction here uses weekday means pooled across regimes; NB04 finds the weekly pattern changed across regimes (the Friday dip deepened after COVID). NB04 reruns the break search with a regime-aware weekday correction and recovers the same dates, and NB03's

regime-aware robustness section carries the same follow-through for its variance tests.